UTN FRC · INGENIERÍA Y CALIDAD DE SOFTWARE · 2026

Desplegar sin detener el streaming.

Un plan de release para microservicios de alto tráfico y millones de usuarios conectados.

CASO 05 · MICROSERVICIOS DE ALTO TRÁFICO

Comisión 4K1 · Grupo 5

Brocanelli · Comas · Koroch· Vilchez · Sanchez· Salinas· Morellato· Tissera· Peñaloza· Imoberdorf· Hermoza· Fassi · Gaitan

Exposición · 03.11.2026



01 / EL PROBLEMA

Un cambio pequeño. Un efecto en cadena.



RIESGOS DEL ESCENARIO

Alta disponibilidad ≠ compatibilidad

Un servicio puede seguir respondiendo y, aun así, romper a sus consumidores por cambiar un contrato.

24/7 | Dependencias | Millones de sesiones

02 / DECISIÓN DE ARQUITECTURA

No todos los despliegues sirven para este caso.

Rolling update

Sustituye réplicas de forma gradual. No controla por sí solo el porcentaje exacto del tráfico.

Base Kubernetes

Blue / Green

Dos entornos listos; permite conmutar, pero el cambio de tráfico puede ser muy amplio.

Alternativa

Canary + GitOps

Exponemos la versión nueva a una fracción del tráfico y la promovemos solo si pasa los controles.

Estrategia elegida

03 / ARQUITECTURA Y ARTEFACTO

Un release reproducible.



ARTEFACTO OBJETIVO

recomendaciones:2.8.0

Imagen OCI por **digest SHA-256** · manifiesto Helm/Kustomize versionado.

REVERSIÓN PREVISTA

recomendaciones:2.7.3

Imagen y configuración estables retenidas y verificadas para volver a dirigir el tráfico.

04 / COMPATIBILIDAD ENTRE SERVICIOS

Primero el contrato. Después el tráfico.

Consumidor Catálogo / app

↓ contrato API ↓

Proveedor Recomendaciones v2.7.3 + v2.8.0

VERIFICACIÓN AUTOMÁTICA

Contract testing · Pact

Reproducir las expectativas reales del consumidor contra la nueva versión del proveedor.

EVOLUCIÓN SEGURA

Expand → migrate → contract

Agregar campos compatibles, migrar consumidores y recién después retirar contratos antiguos.

05 / VALIDACIÓN DE RENDIMIENTO

Antes del canary: probar bajo carga.

01 · PRUEBAS DE RENDIMIENTO

Carga representativa

Simular solicitudes esperadas y medir latencia, errores y capacidad de respuesta.

02 · PRUEBAS DE ESTRÉS

Identificar los límites

Superar gradualmente la carga prevista y comprobar que el servicio se degrade de forma controlada.

03 · TRÁFICO DUPLICADO

Shadow deployment

Copiar solicitudes reales a la nueva versión; solo la versión estable responde al usuario.

04 · DECISIÓN

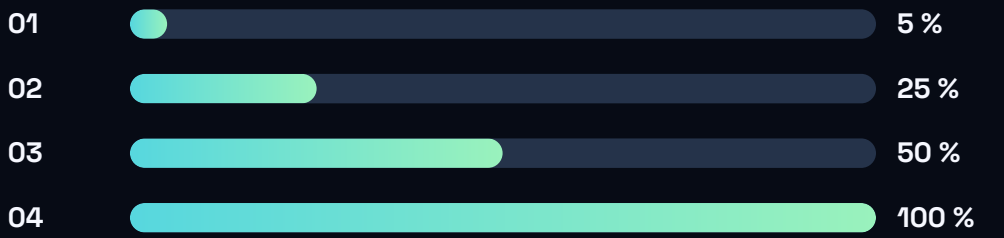
Validar antes de avanzar

Iniciar el canary cuando los resultados cumplan los criterios definidos para el release.

Sommerville, 9.ª ed., § 8.3.3 · Pressman, 7.ª ed., §§ 17.7.3–17.7.4 · Argo Rollouts

06 / LIBERACIÓN PROGRESIVA

El tráfico se gana por etapas.



PUERTA DE AVANCE POR ETAPA

Medir → decidir → promover

Pausa y análisis comparando Canary contra versión estable. La promoción exige señales operativas y funcionales aceptables.

Si falla un control: detener y abortar el Canary.

07 / CONTROL DE CALIDAD EN PRODUCCIÓN

El monitoreo decide.

5xx **Errores de API**
Tasa de fallos Canary vs. estable.

p95 **Latencia**
Tiempo de respuesta bajo carga real.

↗ **Métricas de negocio**
Recomendaciones servidas y reproducción iniciada.

✓ **Smoke tests**
Catálogo, recomendación, historial y reproducción.

Promover
Sin regresión relevante respecto del grupo estable y pruebas post-release satisfactorias.

Abortar + rollback
Error, latencia o funcionalidad fuera de umbrales acordados; retornar tráfico a 2.7.3.

Prometheus · Grafana · OpenTelemetry · Argo Rollouts

08 / PLAN DE RELEASE · OPERACIÓN

Responsables. Controles. Plan de retorno.

- ✓ QA + contratos
- ✓ Capacidad + réplicas
- ✓ Secrets + permisos
- ✓ Observabilidad
- ✓ Versiones fijadas
- ✓ Rollback probado
- ✓ Aprobación release
- ✓ Aviso a soporte

CONTINGENCIA

Abortar Canary → 0 % tráfico nuevo

Conservar réplicas 2.7.3, verificar recuperación, abrir incidente si persiste la falla y reanudar solo tras análisis.

Dev / QA | DevOps / SRE | Release owner

09 / INTEGRACIÓN DE PRÁCTICAS

La entrega segura es un sistema.

01

SCM

Git versiona código, configuración, manifiestos, historial y responsables de cada cambio.

02

DevOps

Desarrollo y operaciones comparten automatización, observabilidad y respuesta ante incidentes.

03

Prácticas continuas

CI verifica y construye; CD prepara y entrega; GitOps reconcilia el estado aprobado.

SEGURIDAD TRANSVERSAL · DEVSECOPS

Escaneo de dependencias e imágenes · firmas y procedencia · secretos externos · RBAC · políticas de admisión

10 / CONCLUSIÓN DEL GRUPO

No desplegamos versiones. Desplegamos confianza.

Compatibilidad antes del release. Evidencia para avanzar. Reversión preparada.

MENOR IMPACTO · MAYOR TRAZABILIDAD · DISPONIBILIDAD

5→100%

Cambio progresivo, sin interrupción planificada.

APORTE ORIGINAL DEL GRUPO

Dos puertas de seguridad

Compatibilidad para entrar a producción + **observabilidad** para ampliar tráfico.

FUENTES Y AGRADECIMIENTOS

Referencias consultadas.

CÁTEDRA Y FUNDAMENTOS

ICS UTN FRC · Consigna de Trabajo de Investigación 1, 2026.

Sommerville, I. *Ingeniería de Software*, 9.ª ed., § 8.3.3.

Pressman, R. *Ingeniería del Software: Un enfoque práctico*, 7.º ed., §§ 17.7.3–17.7.4.

ENTREGA Y OPERACIÓN

Google SRE Workbook · Canarying Releases

sre.google/workbook/canarying-releases

Kubernetes · Update a Deployment Without Downtime

kubernetes.io/docs

IMPLEMENTACIÓN PROGRESIVA

Argo Rollouts · Canary / Analysis

argoproj.github.io/argo-rollouts

Argo Rollouts · Traffic Mirroring

argoproj.github.io/argo-rollouts/features/traffic-management

Pact · Consumer Driven Contracts

docs.pact.io

HERRAMIENTAS DE IA

OpenAI · ChatGPT (GPT-6), apoyo en estructuración, edición y desarrollo del recurso visual HTML, octubre de 2026.

Ilustraciones: diagramas SVG/CSS propios.